

PROGRAMMAZIONE 2

a.a. 2025-2026

leggere/scrivere su file

FILE?

Unità in cui, agli occhi dell'utente, è organizzata la memoria persistente(*) di un computer

Un file contiene una collezione di dati correlati, ad esempio:

- ★ Immagini (.jpg, .gif, .png ...),
- ★ video (.mov ...),
- ★ testi e programmi (.txt, .c, .java, ...),
- ★ queste slide,
- ★ dati

(*) persistente: che sopravvive allo spegnimento del computer

ELABORAZIONE DI FILE, ESEMPI

1. **gcc** è un programma che prende in input uno o più file sorgenti e produce un file nuovo
2. un **editor** è un programma che può creare un file nuovo o modificarne uno esistente
3. una **galleria di immagini** è un programma che permette di scorrere e visualizzare file
4. un **player** video è un programma che mostra file video
5. un **browser** è un programma che mostra file .html

ESEMPIO PIÙ SEMPLICE – CASO DI STUDIO

Un **termometro** registra in un file le temperature delle ore 12:00 di tutti i giorni della settimana

Un **cruscotto** legge tali valori e calcola temperatura media, minima e massima della settimana

ESEMPIO PIÙ SEMPLICE – CASO DI STUDIO

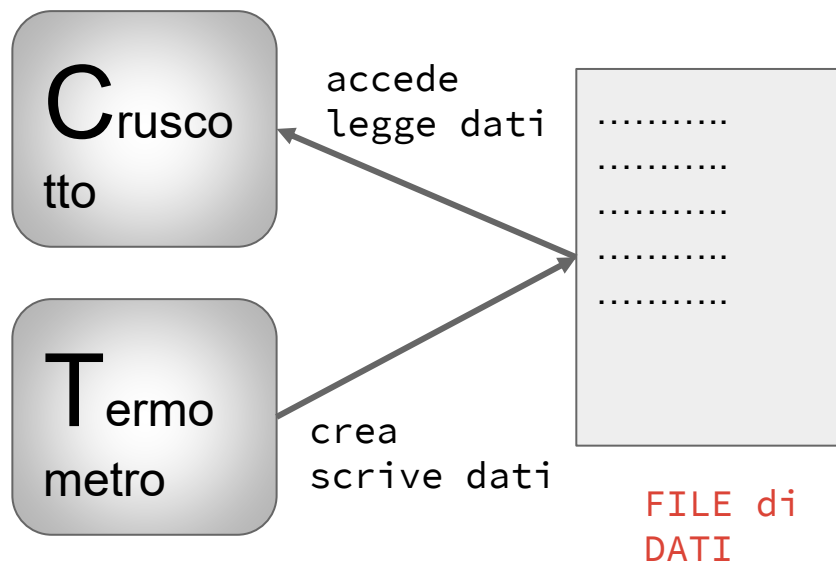
Un **termometro** registra in un file le temperature delle ore 12:00 di tutti i giorni della settimana

Un **cruscotto** legge tali valori e calcola temperatura media, minima e massima della settimana

PROBLEMI

- ★ come si crea un file?
- ★ come si accede a un file?
- ★ come si legge da un file?
- ★ come si scrive in un file?

PROGRAMMI CHE ELABORANO DATI



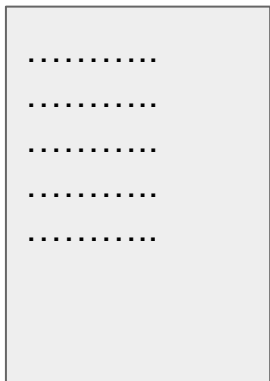
PROGRAMMI

I FILE SOLO RISORSE

UN **PROGRAMMA** DEVE:

- 1) OTTENERNE L'ACCESSO
- 2) USARLO
- 3) RILASCIARE IL FILE

FILE E DATI



FILE

I **DATI** SONO ORGANIZZATI SECONDO UN **FORMATO**

PER ESEMPIO NUMERI INTERI SEPARATI DA SPAZI

PER ELABORARE I DATI OCCORRE CONOSCERE IL
FORMATO

CRUSCOTTO E TERMOMETRO **CONDIVIDONO** IL FORMATO

I/O FORMATTATO SU FILE

fopen	ottiene un tipo di accesso al file
feof	test: fine file?
fscanf	legge da un file
fprintf	scrive in un file
fclose	rilascia il file

Libreria **stdio** `#include <stdio.h>`

STANDARD INPUT OUPUT

fopen	<code>FILE *fopen(const char *pathname, const char *mode);</code>
feof	<code>int feof(FILE *stream);</code>
fscanf	<code>int fscanf(FILE *stream, const char *format, ...);</code>
fprintf	<code>int fprintf(FILE *stream, const char *format, ...);</code>
fclose	<code>int fclose(FILE *stream);</code>

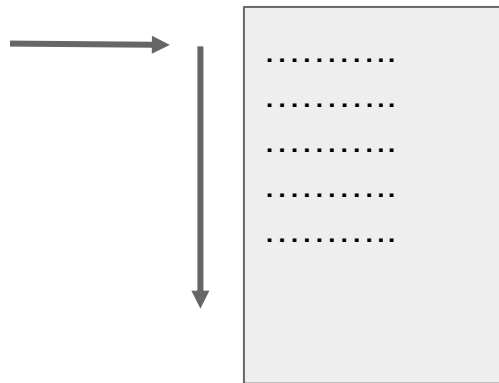
FILE *

- ★ **FILE** è un tipo di dato definito in `stdio.h`
- ★ **fopen** apre un file in una modalità (**r**, **w**, ...)
- ★ L'apertura produce un **flusso di dati** (**FILE***) la cui **direzione** (da file, verso il file) dipende dalla modalità di apertura (**r**, **w**, ...)
- ★ **Es. fopen("file", "w")** produce un flusso di dati verso il file che non può essere usato per leggere i contenuti

FOPEN R (READ)

Apertura in lettura:

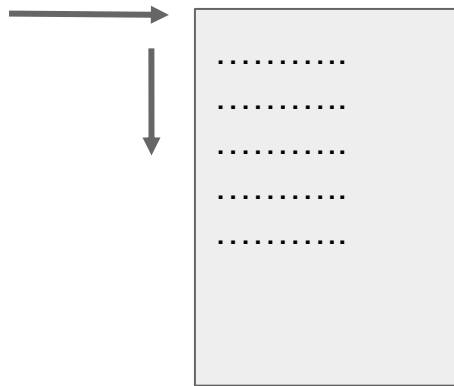
- ★ posizione del FILE*: primo byte del file
- ★ lettura sequenziale
- ★ ogni lettura sposta il puntatore automaticamente del numero di byte letti



FOPEN W (WRITE)

Apertura in scrittura:

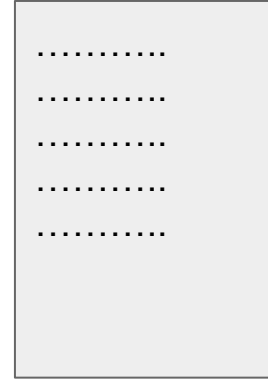
- ★ posizione del FILE*: primo byte del file
- ★ scrittura sequenziale
- ★ eventuali contenuti precedenti vengono cancellati
- ★ ogni scrittura sposta il puntatore automaticamente
- ★ fclose ha per argomento il FILE* da chiudere



FCLOSE

un file aperto va rilasciato:

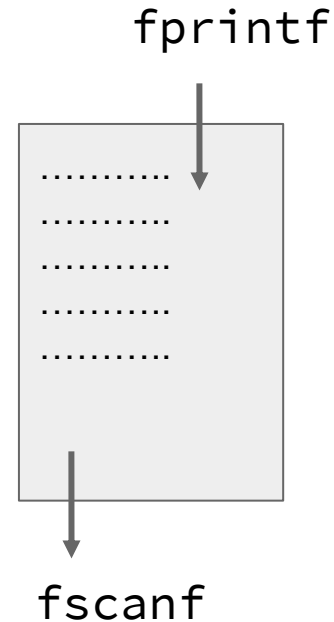
- ★ i file sono risorse
- ★ vanno richiesti (fopen) e vanno obbligatoriamente rilasciati (**fclose**)
- ★ non chiudere il file può creare inconsistenza dei dati
- ★ il programmatore deve ricordarsi di invocare fclose nei punti opportuni del programma



FSCANF, FPRINTF

lettura/scrittura:

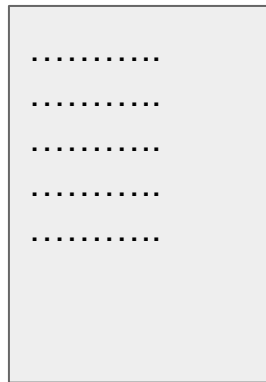
- ★ fscanf (lettura) e fprintf (scrittura) permettono di operare su un FILE*
- ★ entrambe prevedono la specifica di un formato, una stringa che indica cosa leggere/scrivere
- ★ il formato è come per printf/scanf
- ★ seguono ulteriori parametri in numero variabile



feof: C'È QUALCOSA?

test:

- ★ i file hanno lunghezza variabile
- ★ quando leggiamo occorre uno strumento per capire se siamo giunti alla fine del file
- ★ feof è un test su un FILE*
- ★ eof = end of file



fscanf

TERMOMETRO

```
#include <stdio.h>
```

```
...
```

```
FILE * fp = fopen("dati.txt", "w");
```

modalità: scrittura



pathname del file



- se esiste viene “aperto” cioè reso accessibile
- se non esiste viene creato,
- in caso di errore restituisce NULL
- fp: permetterà tutte le operazioni sul file

ESEMPIO: CRUSCOTTO

```
#include <stdio.h>
```

```
...
```

```
FILE * fp = fopen("dati.txt", "r");
```

modalità: lettura



pathname del file,



- se non esiste → errore → NULL
- se esiste viene “aperto” cioè reso accessibile
- fp: permetterà tutte le operazioni sul file

ESEMPIO: LETTURA DI UN NUMERO INTERO

```
#include <stdio.h>
```

```
...
```

```
FILE * fp = fopen("dati.txt", "r");
```

```
fscanf(fp, "%d", &x); // x vrb intera
```

```
fclose(fp);
```

ESEMPIO CRUSCOTTO: GUSCIO

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;    // permetterà l'accesso ai dati
```

```
    ... altre vrb ...
```

```
    fp = fopen("dati.txt", "r"); // apertura file di dati
```

```
    if (fp != NULL) {           // se riuscita
```

```
        ... codice ...         // uso il file
```

```
        fclose(fp);           // rilascio il file
```

```
        ... codice ...         // altro codice
```

```
    }
```

```
    else ... codice errore     // gestisci errore
```

```
}
```

ESEMPIO CRUSCOTTO

```
if (fp != NULL) {  
    while (!feof(fp)) { // finché non ho esaurito i contenuti  
        ... leggi dato  
        ... elabora dato  
    }  
    fclose(fp);  
}
```

feof restituisce un valore diverso da 0 una volta raggiunto l'ultimo byte del file, altrimenti restituisce 0 che in una condizione equivale a falso

ESEMPIO CRUSCOTTO

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        fscanf(fp, "%d", &temp);  
        ... elabora dato  
    }  
    fclose(fp);  
}
```

temp è una nuova vrb intera
da dichiarare

dopo la lettura temp conterrà
il valore letto nel file

fp viene aggiornato automaticamente

ESEMPIO CRUSCOTTO

```
if (fp != NULL) {  
    while (!feof(fp)) {  
        fscanf(fp, "%d", &temp);  
        if (temp < min) min = temp;  
        if (temp > max) max = temp;  
        // ... come calcolare la media?  
    }  
    fclose(fp);  
}
```

min e max sono nuove vrb da dichiarare e inizializzare opportunamente: come?

cosa occorre per il calcolo della media?

ESEMPIO CRUSCOTTO

```
#include <stdio.h>
```

```
#define MAXN 1000
```

```
int main() {
```

```
    FILE *fp;
```

```
    int min=MAXN, max=-MAXN, cont=0, sum=0;
```

```
    fp = fopen("dati.txt", "r");
```

```
    if (fp != NULL) {
```

```
        ...    cont++; sum+=temp;    // ... sta per aggiornam. min e max
```

```
        fclose(fp);
```

```
        printf("min=%d, max=%d, avg=%d", min,max,sum/cont);
```

```
    }
```

```
    else printf("Errore: file inesistente");
```

```
}
```

ESEMPIO TERMOMETRO

```
#include <stdio.h>
```

```
int main() {
```

```
    int temp; int i;
```

```
    FILE *fp = fopen("dati.txt", "w"); // apertura in scrittura
```

```
    for (i=0;i<7;i++) {
```

```
        // simuliamo la registrazione della temperatura per una settimana
```

```
        printf("%d, temp? ", i);
```

```
        scanf("%d", &temp);
```

```
        fprintf(fp, "%d ", temp); // scrive nel file
```

```
    }
```

```
    fclose(fp);
```

```
}
```


ESEMPIO, NOTE

- ★ **il codice è incompleto:** manca la gestione del NULL, completate voi
- ★ **quando può fallire l'apertura in scrittura?**
- ★ compilare il codice e **fare più esecuzioni**, provando casi diversi
- ★ come si potrebbe **generalizzare il termometro** così che a ogni esecuzione possa inserire un numero di temperature/cicli differente? Il cruscotto va modificato?

EXTRA

- ★ `fopen` ha numerose altre modalità fra cui “a” che permette l’apertura in “append”: il puntatore si posiziona in fondo ai contenuti attuali per aggiungere dati a quelli già esistenti
- ★ il formato di `fprintf` e `fscanf` può essere articolato, approfondire da soli

FGETS

- ★ `char *fgets (char *string, int n, FILE *stream);`
- ★ La funzione `fgets()` legge i caratteri dal flusso *corrente* posizionandosi fino al primo carattere di nuova riga incluso (`\n`), fino alla fine del flusso o fino a quando il numero di caratteri letti non è uguale a $n-1$, a seconda di quale dei due si verifica per primo. La funzione `fgets()` memorizza il risultato in *stringa* e aggiunge un carattere null (`\0`) alla fine della stringa. La *stringa* include il carattere di nuova riga, se letto. Se n è uguale a 1, la *stringa* è vuota.

```
char buffer[100];  
printf("Enter text: ");  
  
// Safely read up to 99 characters  
if (fgets(buffer, sizeof(buffer), stdin) != NULL) {  
    printf("You entered: %s", buffer);  
}
```

ESEMPIO RIASSUNTIVO

vediamo insieme un programma che, a scelta dell'utente, legge oppure scrive uno stesso file

coniughiamo: interazione con utente (input da tastiera, output su terminale) con input/output su file

ESEMPIO RIASSUNTIVO

```
#include <stdio.h>
#define MAX 20

int main() {
    int scrivi; // il programma deve, a scelta dell'utente,
                // leggere o scrivere un file

    FILE *fp;
    char datiS[MAX];
    int datiN;

    printf("\n vuoi leggere o scrivere? 1: scrivere, 0: leggere ");
    scanf("%d", &scrivi);

    if (scrivi) { // 1: vero, 0: falso
        ...
    }
    else {
        ...
    }
}
```

ESEMPIO RIASSUNTIVO

```
{
...
    if (scrivi) {
        fp = fopen("dati.txt", "w"); // chiedo accesso in w al file
        if (fp) { // se ottenuto
            fprintf(fp, "nome: Anna id: 123\n"); // scrivo
            fclose(fp); // rilascio il file
        }
    }
    else {
        fp = fopen("dati.txt", "r"); // chiedo accesso in r al file
        if (fp) { // se ottenuto
            fscanf(fp, "nome: %s id: %d\n", datiS, &datiN); // leggo
            printf("\nHo letto:%s %d\n", datiS, datiN);
            fclose(fp); // rilascio il file
        }
    }
}
```

ESEMPIO RIASSUNTIVO

```
{
...
    if (scrivi) {
        fp = fopen("dati.txt", "w"); // chiedo accesso in w al file
        if (fp) { // se ottenuto
            fprintf(fp, "nome: Anna id: 123\n"); // scrivo
            fclose(fp); // rilascio il file
        }
    }
    else {
        fp = fopen("dati.txt", "r"); // chiedo accesso in r al file
        if (fp) { // se ottenuto
            fscanf(fp, "nome: %s id: %d\n", datiS, &datiN); // leggo
            printf("\nHo letto:%s %d\n", datiS, datiN);
            fclose(fp); // rilascio il file
        }
    }
}
```

LETTURA FORMATTATA

```
fscanf(fp, "nome: %s id: %d\n", datiS, &datiN);
```

- ★ il formato specifica che ci aspettiamo che la stringa e il numero da leggere siano inseriti in un testo che segue il **pattern** indicato

nome: NOME-DA-LEGGERE id: NUM-DA-LEGGERE

- ★ La parte di pattern esplicito non viene memorizzata
- ★ **Provare più esecuzioni:** cosa succede se i dati non corrispondono al pattern atteso?

ESERCIZIO

Scrivere un programma che **legge dati da un file** contenente cifre scritte come numeri (0, 1, 2 ... 9) e **produce un secondo file** contenente le stesse cifre scritte come parole (zero, uno, due, ... nove)

Tutto il contenuto del primo file (opportunamente codificato) va riversato nel secondo file